TelcoBench Evaluation Framework

Technical description of the reproducible generation, rubric-evaluation, and reporting workflow for telecommunications and network-infrastructure language models

15 October 2026

Based on the repository README, data card, judge protocol, evaluation framework design, manifest, rubric configuration, and implementation modules.

Contents

1. Executive summary

2. Benchmark contract and identity

3. Frozen evaluation plan

4. Generation workflow

5. Evidence gate and judging workflow

6. Aggregation and comparison

7. Lifecycle management and audit evidence

8. Interpretation and limitations

1. Executive summary

TelcoBench is a reproducible evaluation harness for language models that answer telecommunications and network-infrastructure questions. Its central design principle is evidence preservation: the framework validates a frozen set of prompts, records every generation attempt, evaluates preserved successful answers with a versioned rubric, and produces summaries only when coverage and provenance checks are satisfied.

The benchmark contains held-out, route-labelled questions, not reference answers or executable correctness oracles. Its results are therefore rubric-based quality assessments rather than reference-grounded accuracy, pass rate, or proof of operational correctness. A valid result pairs a 0–25 LLM-judge score with evidence about the generation and judging conditions that produced it.

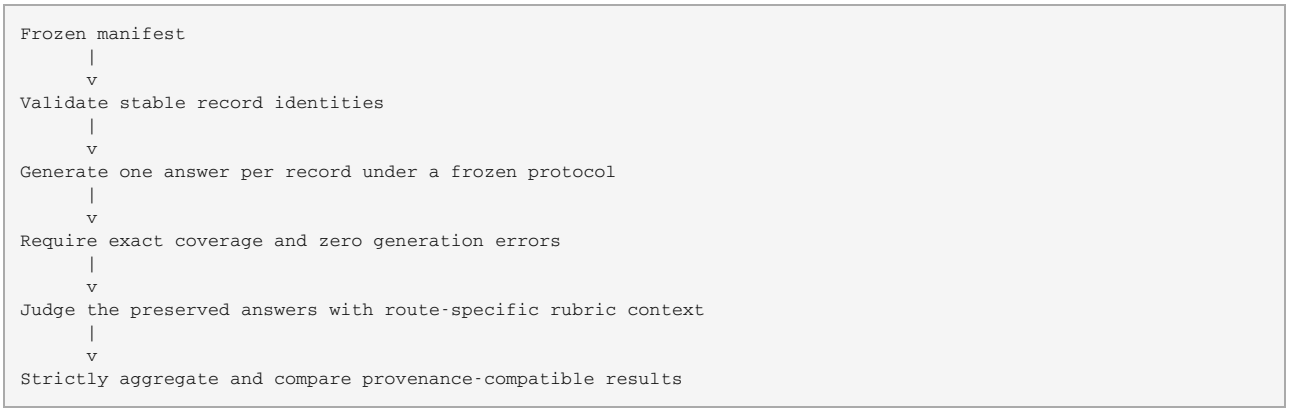


Figure 1. The TelcoBench evaluation flow.

The framework has four functional layers. The contract layer validates benchmark inputs and establishes stable identities. The generation layer supports local Hugging Face/PEFT models and OpenAI-compatible endpoints. The judging layer evaluates saved answers against five 0–5 dimensions. The control and reporting layer manages workers, schedules eligible GPUs, gates stage transitions, and creates comparison reports.

2. Benchmark contract and identity

`datasets/MANIFEST.json` is the root of trust for the benchmark. Before generation, the framework validates each selected route file against its expected SHA-256 digest and record count. It also verifies required fields, confirms that the route label in each record matches the manifest route, and checks uniqueness. These checks prevent an evaluation from silently using modified, incomplete, or mislabelled inputs.

The manifest defines 2,850 prompts across 14 routes. The routes cover BGP, OSPF, IS-IS, MPLS VPN, subscriber management, quality of service, IP routing, multicast, Linux diagnostics, networking fundamentals, Layer 2 switching, PON/optical access, and Wi-Fi gateways. Route labels define the canonical benchmark scope; they do not imply that real-world networking incidents can be assigned to only one route.

Required field	Role
`prompt`	The question given to the evaluated model and, later, to the judge together with the saved answer.
`sample_id`	A source-local identifier. It is not unique across the entire benchmark.
`teacher_routing_key`	The canonical route label, which must match the route declared in the manifest.

The durable identity of an example is the pair `(route, sample_id)`, rendered internally as `route:sample_id`. This identity connects input records, generated answers, and judge records. Using the pair rather than `sample_id` alone prevents a score for one route from being attributed to an identically named sample in another route.

> **Scope of the measurement.** Because the benchmark supplies prompts and route labels but no reference answers or executable oracles, TelcoBench does not produce automatic correctness labels. Its purpose is reproducible generation coverage and structured quality evaluation.

3. Frozen evaluation plan

A YAML configuration describes a concrete evaluation plan. It identifies the benchmark manifest, generation-prompt protocol, output root, maximum output length, model-loading policy, evaluated targets, artifact identities, and judge/rubric configuration. Environment variables may supply local paths, endpoint details, and credentials. If a referenced variable is not resolved, loading the configuration fails, keeping unpublished environment details out of versioned configuration while avoiding implicit defaults.

For managed local runs, model weights and adapters are checked against configured SHA-256 digests before a worker starts. The digest cache may be reused only if device, inode, size, and nanosecond modification time still match. Remote targets instead persist their endpoint and configured model identifier in the run evidence. Resume is fail-closed: the existing metadata must match the manifest hash, selected routes, prompt-protocol hash, model, adapter, and decoding settings before prior rows can be reused.

4. Generation workflow

Generation starts by validating the manifest, loading the versioned prompt protocol, selecting routes, and calculating the expected identity set. Before model loading, the runner writes `run.json` and `generation-progress.json`. This makes initialization visible to the monitor and separates a slow model load from a missing or failed worker.

Every completed attempt is appended immediately to `generations.jsonl`. A successful row (`status: "ok"`) retains the route, sample ID, original prompt, answer, token counts, and prompt hash. A failed row (`status: "error"`) retains the route, sample ID, prompt, exception type, and error message. Failures are evidence and are never removed merely to improve reported coverage. At completion, `coverage.json` states the expected count, total rows, unique identities, missing identities, successful rows, and error count.

Prompt and decoding protocol

The primary deployment-like condition is the neutral prompt protocol: every example receives the same system prompt and does not receive its route label. A separate route-conditioned protocol exists for diagnosis. It names the evaluated expertise and must not be reported as deployment-like performance; comparisons that use it should apply it to every evaluated model.

The local MOPD-compatible path uses CUDA, BF16 model loading, `fix_mistral_regex=True` tokenizer repair, and the model's native `apply_chat_template(..., add_generation_prompt=True)` interface. Its standard protocol uses greedy decoding: `do_sample=false`, temperature 0, and `top_p=1`. Sampling-only options are removed when greedy generation is requested.

Model backends

Backend	Framework behavior
Local Hugging Face / PEFT	Loads a local model, optionally loads a LoRA adapter under an explicit `student` name, selects it, and enables it. The run fails if PEFT exposes no matching LoRA layers or reports disabled layers, guarding against a supposed student result that is actually the base model.
OpenAI-compatible endpoint	Verifies that the configured model appears in the provider catalog and submits non-streaming chat-completions calls using the same system/user structure. Calls have bounded retry with a five-minute delay. An explicitly recorded fallback may retry an empty, length-truncated response with provider-specific template arguments.

5. Evidence gate and judging workflow

Judging is normally permitted only when generation has no missing manifest records, no generation-error rows, and a unique generated identity set exactly equal to the expected identity set. The `--allow-incomplete` option is available for controlled diagnostics, but weakens that requirement and is not appropriate for a complete comparison. The managed pipeline enforces the strict form: a target with missing, erroneous, or inconsistent generation evidence fails rather than proceeding to final judging.

The judge consumes frozen `generations.jsonl` data; it does not regenerate answers. A `judge-run*.json` plan records hashes of the generation evidence and judge protocol, selected routes, progress, and completion state. Thus the judgment is bound to the specific answers that were generated under the recorded conditions.

Each pending successful answer is evaluated with the versioned global judge protocol, route label, rendered route-specific rubric context, original question, and frozen model answer. A route rubric is mandatory for every pending route. Missing or empty route context stops judging before any provider request. The current built-in judge transport is Anthropic-compatible and uses temperature 0.

Dimension	Range	Meaning
Factual	0–5	Technical correctness within the relevant route domain.
Completeness	0–5	Coverage of the material aspects of the question.
Terminology	0–5	Correct standards, commands, and domain vocabulary.
Hallucinations	0–5	Absence of technically false or fabricated claims.
Usefulness	0–5	Practical value to a network engineer.

The judge must return a JSON array aligned to the input batch. The runner rejects malformed responses, a wrong number of objects, invalid dimension keys, non-integer scores, and values outside 0–5. A valid vector has a total from 0 to 25. Parsed values are appended to `judgments.jsonl`; raw provider responses go to `judge-raw.jsonl`; batches that exhaust retry attempts go to `judge-failures.jsonl` and remain unscored for a later resume.

6. Aggregation and comparison

Aggregation strictly joins judgments to successful generations by stable identity. It rejects unknown or duplicate judgments, invalid dimensions, and stored totals that disagree with the sum of the five dimensions. The scorer reports `generated_ok`, `scored`, and `unscored_generated`, making score coverage visible rather than allowing it to be hidden in an average.

The micro mean is the mean total across all scored examples. The macro route mean is the unweighted mean of the per-route means. The former reflects average quality over the whole benchmark and gives larger routes more influence; the latter gives each route equal influence and is useful because the benchmark is route-imbalanced.

```
micro mean       = mean(total score for every scored example)
macro route mean = mean(mean total within each scored route)
```

Before it produces a report, the framework requires complete judgment coverage. For the standard comparison, it also requires both `cpt_base` and `mopd_student_step_2000`, equal route coverage for those targets, and exactly identical judge provenance. The resulting comparison provides micro, macro, and per-route `MOPD − CPT` deltas. Route-specialist targets may be combined only when their route sets do not overlap.

7. Lifecycle management and audit evidence

The control layer manages only workers launched by the package. Its control file records PID, Linux process start-time ticks, full command, log path, and start time. Status checks both the PID start time and command line to avoid confusing a reused PID with an evaluator. Stop sends a signal only to the process group recorded by a prior start.

The `start-all` coordinator dynamically schedules local GPU targets. It polls `nvidia-smi`, admits only GPUs beneath configured idle memory and utilization thresholds, and assigns at most one active evaluator to a GPU. A generation target becomes eligible for judging only after its coverage is valid. Network-bound judging may overlap further GPU generation, while the coordinator limits route-judge concurrency to protect the judge gateway. Remote-only targets bypass GPU admission. The TUI is intentionally read-only and derives its state from persisted evidence files.

Artifact	Contents
`run.json`	Generation protocol, hashes, model/adapter identity, runtime, decoding, and prompt hashes.
`generation-progress.json`, `generations.jsonl`, `coverage.json`	Live progress, append-only generation outcomes, and final coverage/error evidence.
`judge-run*.json`, `judge-raw.jsonl`, `judge-failures.jsonl`, `judgments.jsonl`	Judge plan, raw replies, exhausted batches, and parsed rubric vectors.
`score-summary*.json`, `comparison-report.json`	Strict aggregate scores, judge provenance, and final cross-target result.

8. Interpretation and limitations

TelcoBench results should be interpreted as comparisons of answer quality under a named manifest, prompt protocol, model or adapter identity, judge identity, and configuration. A complete report should state whether it used the neutral or route-conditioned protocol, disclose generation and judgment coverage, include errors, and show both micro and macro results when relevant.

A high score does not demonstrate operational correctness or safety. The benchmark does not have reference answers or executable oracles, the routes are imbalanced, and it does not claim complete coverage of the networking domain. The data card also notes that `linux_tool_calling` and `linux_tool_agentic` are separate compatibility labels but not an independently constructed pair of held-out tasks.

For stronger research or product claims, the repository recommends a blinded, stratified audit by networking subject-matter experts. That study should report agreement between human evaluation and the judge by route and by dimension. It should also address remaining provenance, privacy, duplicate/near-duplicate, and route-label validation work documented in the data card.

> **In short:** TelcoBench is designed to make the generation and LLM-judging process reproducible and auditable. It is not designed to convert a question-only benchmark into an automatic proof that an answer is correct.